

Data Storage and Functions within
Student Medical Centre - University of Moratuwa

Group 7

IN1111 - Data Structures and Algorithms 1
Practical Assignment

244021L K T H A Ariyawansa
244125J M M S S Mendis
244002F H A V Abeywardena
244157H M T N A. Perera
244170P K H S S Ranbandara
244171U K L G V Randinu
244084F D L S R Jayasekara

Table of Contents

Table of Contents.....	2
Introduction.....	3
System Design.....	4
Implementation of Instances and Sample Outputs.....	5
1. Register New Students in the Medical Centre.....	5
Implementation.....	5
Sample Outputs.....	5
2. Organizing Patient Appointments Based on Urgency and Arrival.....	6
Implementation.....	6
Sample Outputs.....	6
3. Recording Patient Treatment Time and Cost.....	7
Implementation.....	7
Sample Outputs.....	7
4. Retrieving Latest Lab Report.....	8
Implementation.....	8
Sample Outputs.....	8
5. Maintain Details of National Hospitals Within Proximity.....	9
Implementation.....	9
Sample Outputs.....	9
6. Identifying the Most Frequent Medical Issues Among Students.....	10
Implementation.....	10
Sample Outputs.....	10
7. Organizing Pharmaceutical Suppliers by Cost and Distance.....	10
Implementation.....	11
Sample Outputs.....	11
Discussion.....	12
5. Conclusion.....	14

Introduction

The Student Medical Centre at the University of Moratuwa is an essential service that uplifts student and staff welfare. As the only medical centre for over 10, 000 undergraduates, it is a well-organized and highly efficient structure that handles large quantities of data on a regular basis. Efficient organization of this data is necessary to ensure there is a smooth patient flow and accurate records are maintained.

This project presents a console-based C application. It is designed for the operations of the University of Moratuwa medical centre using appropriate data structures. The system integrates multiple common occurrences including student registration, appointment handling, treatment tracking, lab report management, nearby hospital referrals, supplier organization, and medical data analysis.

The data structures implemented in this proposal are to further develop the efficiency and data storage of the present system at the University of Moratuwa medical centre.

System Design

The implemented system follows the existing workflow of the university medical centre. Each operation is supported by a data structure chosen according to the nature of the task.

The operational flow:

1. Students are registered in the medical centre database.
2. Patients' appointments are organised based on urgency and arrival time.
3. Treatment details and patient timing information are recorded.
4. Doctors access the latest lab reports during consultations.
5. Patients requiring specialised care are referred to nearby hospitals.
6. Medical trends are analysed by identifying frequent illnesses.
7. Pharmaceutical suppliers are organised to support medicine procurement.

Different data structures are integrated into one system:

- **Doubly Linked List** – Student registration records
- **Queue** – Appointment management
Treatment tracking
- **Stack** – Lab report retrieval
- **Array** – Hospital information management
- **Singly Linked List** – Medical issue frequency analysis
– Supplier organization

This unified design ensures efficient handling of data that is both dynamic and fixed in terms of size.

Implementation of Instances and Sample Outputs

1. Register New Students in the Medical Centre (Proposed Data Structure - Doubly Linked List)

Implementation

A Doubly Linked List is used to store student records in sorted order based on index number.

Each node contains:

- Index Number
- Student Name
- Degree
- Batch
- Next Pointer
- Previous Pointer

Operations:

- Insert the student in the sorted position
 - Create a new node with student details
 - Traverse the doubly linked list to find the correct position based on the index number
 - Insert the student node in its sorted position, maintaining the order of index number
- Search student by index or name
 - Traverse a list to locate a student by index number or name
- Display registered students
 - Output the list of registered students in ascending order of index numbers

Sample Outputs

Student Registered Successfully

Index: 244001F | Name: Amal Perera | Information Technology | 24 batch

Index: 244023L | Name: Nimal Silva | Information Technology | 24 batch

Index: 245057K | Name: Kasun Jayasekara | Artificial Intelligence | 24 batch

2. Organizing Patient Appointments Based on Urgency and Arrival (Proposed Data Structure - Queue)

Implementation

This system organizes patients into two categories: Normal and Urgent, based on their condition. By using two separate queues, urgent patients are prioritized while maintaining a first-come, first-served order within each category. When the doctor is available, patients from the urgent queue are served first, before the normal patients.

Two Queues are maintained:

- Urgent Queue
- Normal Queue

Patients are enqueued according to priority and dequeued when the doctor becomes available.

Sample Outputs

Patient Added: Token 01 | Amal | Normal

Patient Added: Token 02 | Nimal | Urgent

Urgent Queue: [Token 02]

Normal Queue: [Token 01]

Next Patient: Token 02 – Nimal

3. Recording Patient Treatment Time and Cost

(Proposed Data Structure - Queue)

Implementation

A Queue stores treatment session details. When a student comes into the medical centre, they are given a token number, and the time at which they are accepted as a patient is recorded. If any medicine or treatment is given to the student, the total cost is recorded. Those details are linked to the student's detail records permanently and to the token temporarily. (Student records will be updated separately, as only token details are not relevant to this calculation.) When the student is discharged, the discharge time is recorded as well. The token numbers are reused every day, such that there is no connection between the students and the token numbers they receive. Only the in and out times and treatment issued will be linked to the token number for that day. Patients are dequeued after discharge and tokens are reused.

Queue node includes:

- Token Number
- Arrival Time
- Total Cost
- Discharge Time

Sample Outputs

1. Enter New Patient
2. Process Existing Patient
3. Show Averages & Exit

Enter choice: 1

Enter arrival time (HH:MM): 12:00

Token issued: 1

Enter choice: 2

Enter token number: 1

Enter departure time (HH:MM): 12:45

Enter treatment cost: 1200

Patient 1 discharged.

Enter choice: 3

Average Time per Patient: 00:45

Average Cost per Patient: 1200.00

4. Retrieving Latest Lab Report

(Proposed Data Structure - Stack)

Implementation

A Stack temporarily stores lab reports during doctor sessions. Patient lab reports such as blood tests, X-rays, and scan results are maintained in a centralized database in the medical center. Doctors usually review the most recent reports first because they reflect the patient's current medical condition. To efficiently manage the viewing process, the medical center temporarily organizes retrieved reports for doctors's use.

Hospital Database - permanently stores all lab reports

Stack (Temporary Memory) - holds reports during doctor's session

Doctor Interface - displays reports to the doctor

Operations:

- PUSH – load report
- PEEK – view latest report
- POP – remove reviewed report
- isEmpty – check availability

Sample Outputs

Report Loaded: Blood Test

Report Loaded: X-Ray

Report Loaded: MRI Scan

Top -> MRI Scan

Latest Report Viewed: MRI Scan

5. Maintain Details of National Hospitals Within Proximity

(Proposed Data Structure - Array)

Implementation

The university medical centre may need to transfer patients to nearby national hospitals when specialised treatment, advanced procedures, or additional capacity are required. To support fast and informed transfer decisions, the system maintains a structured record of all hospitals within proximity. Hospital records are stored using an Array.

Each hospital entry contains identification details, location, and supported specialisations. This enables efficient identification of the most suitable hospital for patient transfers based on requirements.

Each element contains:

- Hospital ID
- Name
- Distance
- Specialisations
- Contact Number

Operations:

1. Insert a new hospital record into the array.
2. Modify fields such as contact number or specialisations.
3. Search Hospitals by specialisation
4. Sort Hospitals by Distance using insertion sort

Sample Outputs

Hospitals Sorted by Distance:

1. Colombo National Hospital – 5km - 041 222 3412
2. Kalubowila Hospital – 8km - 041 223 1583

Search: Cardiology

Search Result: Hemas Hospital Nugegoda - 041 223 3244

6. Identifying the Most Frequent Medical Issues Among Students (Proposed Data Structure - Singly Linked List)

Implementation

Our system maintains a structured record of all reported medical issues. By tracking and analysing the frequency of each issue, common health patterns among students can be observed. A Singly Linked List tracks medical issues and occurrence counts.

Functions:

- Each reported medical issue is recorded
- If the same issue occurs multiple times, its occurrence count is incremented rather than creating duplicate records.
- The system maintains a dynamic collection of medical issues and their frequencies.
- The list of medical issues can be sorted in descending order based on the number of occurrences using a selection sort
- The most frequent medical issues can be identified efficiently after sorting.

Each node contains:

- Issue ID
- Medical Issue Name
- Number of Occurrences
- Next Pointer

Sample Outputs

Fever – 45 cases

Headache – 31 cases

Cold – 27 cases

7. Organizing Pharmaceutical Suppliers by Cost and Distance

(Proposed Data Structure - Singly Linked List)

Implementation

A Singly Linked List maintains supplier records dynamically. The university medical center relies on a consistent supply of medicines and medical consumables from external pharmaceutical suppliers. To ensure cost-effective procurement while accounting for practical constraints on distance, the system maintains an organised list of all registered suppliers. Each supplier entry contains their identification details, average cost per medicine unit, and their distance from the university. The list can be sorted at any time so that the most financially favourable and geographically convenient suppliers are clearly identified.

Node contains:

- Supplier ID
- Supplier Name
- Contact
- Cost per Unit
- Distance

Functions:

- A new supplier is registered and appended to the list.
- The list can be sorted at any time, ordered by cost per unit, with distance used as a tiebreaker when two suppliers share the same cost using bubble sort
- A supplier can be removed from the list when they are no longer active.
- The cheapest supplier can be instantly identified for quick procurement decisions.

Sample Outputs

Suppliers Sorted:

Sumudu Pharmacy – LKR 45/unit – 3km

Moratuwa Medical Suppliers – LKR 48/unit – 5km

Cheapest Supplier: Sumudu Pharmacy

Discussion

Different data structures were selected based on the operational requirements of each functionality within the medical centre system.

A Doubly Linked List was used for student registration because it allows efficient insertion at any position in a sorted list without shifting elements, unlike arrays. Both previous and next pointers enable bidirectional traversal, simplifying searching, updating, and deleting records. A Circular Linked List was avoided as it introduces unnecessary joint nodes, while queues and stacks are unsuitable since student registrations occur randomly and require continuous updates.

For patient appointment management, a Queue was selected due to its First-In-First-Out (FIFO) behaviour. Using two separate queues allows urgent patients to be prioritized while maintaining fairness among patients within the same category, without modifying the basic queue structure.

Another Queue was used for recording treatment time, cost, and token reallocation. Since tokens are reused after patient discharge, FIFO ordering ensures fair and also efficient reassignment while simplifying enqueue and dequeue operations.

A Stack was implemented for lab report retrieval because it follows the Last-In-First-Out (LIFO) principle, so this aligns with medical practice where doctors review the most recent reports first. Reports are considered based on time of issue, allowing immediate access to the latest information.

An Array is chosen for storing nearby hospital details because the dataset is small and relatively fixed, making it simple and efficient to manage. It allows easy access, updates, and traversal of records, while linear search is sufficient for finding hospitals by specialisation. Insertion sort is used to sort hospitals by distance as it is efficient for small or partially sorted data and maintains stability by preserving the order of equal elements. Although its worst-case time complexity is $O(n^2)$, it is acceptable due to the limited number of records.

A singly linked list is suitable for tracking medical issues because it allows dynamic storage as new issues are reported without needing a fixed size. It supports efficient insertions and avoids duplication by updating occurrence counts within existing nodes. Selection sort is used to order issues by frequency, making it easy to identify the most common conditions. Although selection sort is not a stable algorithm, stability does not significantly affect this scenario because each medical issue is uniquely maintained with its own occurrence count, and there is no practical importance in preserving the relative order of issues with equal frequencies. Additionally, its time complexity of $O(n^2)$ is acceptable due to the relatively small number of distinct issues.

Finally, A Singly linked list is suitable for managing pharmaceutical suppliers because it allows dynamic addition and removal of records without requiring a fixed size, making it flexible for changing data. Bubble sort is used to organise suppliers by cost, with distance as a tiebreaker, and it is a stable sorting algorithm, meaning suppliers with equal costs maintain their relative order based on distance. Although its time complexity is $O(n^2)$, it is acceptable due to the relatively small number of suppliers and the simplicity of implementation.

Overall, each data structure was chosen to match real-world medical workflows, ensuring efficiency, simplicity, and practical system design.

Conclusion

This project allowed the team to understand how fundamental data structures and algorithms can be applied to model real operations within the Student Medical Centre at the University of Moratuwa. By designing a system that reflects the medical centre activities such as student registration, patient appointment handling, treatment recording, lab report viewing, hospital referrals, and supplier management, the existing system can be improved.

Source Code

https://drive.google.com/file/d/1Xi9XFpbmFBPhi8xEWK0MyOf44_ttOoGh/view?usp=sharing